

Introduction to causal inference
using machine learning methods

Lab 2:
TMLE for single timepoint interventions

Simulating data

We will use simulated data sets where we know the truth to demonstrate some key ideas.

- We generate a data set of $n = 300$ observations.
- There are two covariates, $W = (W_1, W_2)$.
- The treatment probability is logistic-linear in W_1, W_2 .
- The outcome is generated according to a linear model.
- The true ATE is 0.20.

```
# set a seed for reproducibility
set.seed(212)
# sample size
n <- 300
# W1 has Normal distribution, W2 has Uniform distribution
W1 <- rnorm(n); W2 <- runif(n)
# make a data.frame of covariates
W <- data.frame(W1 = W1, W2 = W2)
# pr(A = 1 | W) is logistic linear in W
g1W <- plogis(-1 + W1 - W2 + W1*W2)
# generate binary treatment
A <- rbinom(n, 1, g1W)
# E[Y | A, W] is logistic linear in A, W
QAW <- plogis(W1 - W2 + A)
# generate outcome by adding random error
Y <- rbinom(n, 1, QAW)
```

The drtmle package

The drtmle package facilitates doubly-robust estimation and inference about average treatment effects. It is available on CRAN and GitHub.

- These slides are based off the package vignette.

Learning objectives for today:

- 1 understanding and executing basic calls to drtmle;
- 2 understanding interface between drtmle and SuperLearner;
- 3 implementing estimators of additional regressions for robust inference;
- 4 implementing drtmle with missing treatment assignments and missing outcomes.

Basic calls to drtmle

A rundown of the most important options for the `drtmle` function:

- `W`: covariates;
- `A`: treatment assignment (can have multiple discrete values);
- `Y`: outcome;
- `family`: `gaussian()` or `binomial()`, description of `Y`;
- `a_0`: `drtmle` will estimate counterfactual mean for all values of `a_0` supplied by user;
- `SL_Q`: super learner library for the outcome regression;
- `SL_g`: super learner library for the propensity score;
- `SL_Qr`: super learner library for residual outcome regression;
- `SL_gr`: super learner library for residual outcome regression;
- `stratify`: should outcome regression pool over levels of `A` (`stratify = FALSE`) or should a separate outcome regression be fit for each level of `A` (`stratify = TRUE`);
- `maxIter`: maximum number of TMLE iterations (sane default is 3, smaller number = faster run);
- `tolg`: truncation level for propensity score (default is 0.01, sanity is context dependent).

Basic calls to drtmle

Let's start by making a simple call to drtmle and parsing the output.

- Get counterfactual mean for both levels of treatment.
- The outcome is binary, so we use `family = binomial()` for SuperLearner wrappers.
- The super learner library for both regressions uses polynomial multivariate adaptive regression splines and logistic regression.
- The super learner library for the residual regression uses same library.
- We fit a single outcome regression using observations with $A = 1$ and $A = 0$.

```
set.seed(123)
fit1 <- drtmle(W = W, A = A, Y = Y, a_0 = c(0,1), family = binomial(),
              SL_g = c("SL.earth", "SL.glm"),
              SL_Q = c("SL.earth", "SL.glm"),
              SL_gr = c("SL.earth", "SL.glm"),
              SL_Qr = c("SL.earth", "SL.glm"),
              stratify = FALSE)

fit1
```

Basic calls to drtmle

Let's start by making a simple call to drtmle and parsing the output.

- `$est` is the estimates for each value of `a_0` (shown as row name).
- `$cov` is the estimated asymptotic covariance matrix divided by sample size.

```
## $est
##
## 0 0.432809
## 1 0.685118
##
## $cov
##              0              1
## 0 0.0011512050 0.0001101087
## 1 0.0001101087 0.0053625842
```

Basic calls to drtmle

`drtmle` also contains a `ci` method for computing doubly-robust confidence intervals.

- `contrast` specifies what quantity you would like a confidence interval for (default is for counterfactual means). Examples of other quantities on subsequent slides.
- `est` specifies which estimator to get a confidence interval for (`drtmle` = doubly-robust confidence intervals; `tmle` = TMLE confidence intervals; `aiptw` = AIPTW confidence intervals).
- `level` determines nominal coverage probability of the interval (default is 95%).

```
ci(fit1)
```

```
## $drtmle
##      est   cil   ciu
## 0 0.433 0.366 0.499
## 1 0.685 0.542 0.829
```

Basic calls to drtmle

If `contrast` is a vector, then `ci` computes confidence interval for the dot product of `contrast` and `fit1$est`.

- E.g., if `contrast = c(1, -1)` then `ci` computes confidence interval for the ATE.

```
ci(fit1, contrast = c(-1, 1))
```

```
## $drtmle
##               est    cil    ciu
## E[Y(1)]-E[Y(0)] 0.252 0.097 0.408
```


Basic calls to drtmle

More generally, `contrast` may be input as a list, which allows the `ci` method to compute confidence intervals of the form

$$f^{-1}\{f(h(\psi_n)) \pm z_{1-\alpha/2}[\nabla f(h(\psi_n))^T \Sigma_n \nabla f(h(\psi_n))]^{1/2}\} , \text{ where}$$

- `f (contrast$f)` is the transformation of the confidence interval,
- `f-1 (contrast$f_inv)` is the back-transformation of the interval,
- `h (contrast$h)` is a contrast of counterfactual means,
- `∇f(h(ψn))(contrast$fh_grad)` defines the gradient of the transformed contrast at the estimated counterfactual means, and
- `Σn` is the doubly-robust covariance matrix estimate (automatically obtained from `drtmle`).

Basic calls to drtmle

For example, we may be interested in the risk ratio $E[Y(0)]/E[Y(1)]$.

- The true risk ratio in the simulated data example is 0.66.

This confidence interval is often computed on the log scale and back-transformed,

$$\exp\left[\log\left\{\frac{\psi_n(1)}{\psi_n(0)}\right\} \pm z_{1-\alpha/2}\sigma_n^{\log}\right],$$

where $\sigma_n^{\log}$ is the estimated standard error on the log-scale.

By the delta method, an estimate of the standard error of the log-risk-ratio is

$$\sigma_n^{\log} = \{\nabla g(\psi_n)^T \Sigma_n \nabla g(\psi_n)\}^{1/2},$$

where $\nabla g(\psi)$ is the gradient of $\log\{E[Y(1)]/E[Y(0)]\}$.

Basic calls to drtmle

This confidence interval can be computed using the following code.

```
riskRatio <- list(f = function(eff){ log(eff) },  
                 f_inv = function(eff){ exp(eff) },  
                 h = function(est){ est[1]/est[2] },  
                 fh_grad = function(est){ c(1/est[1], -1/est[2]) })  
ci(fit1, contrast = riskRatio)
```

```
## $drtmle  
##           est   cil   ciu  
## user contrast 0.632 0.49 0.815
```

Basic calls to drtmle

This method of computing confidence intervals can also be useful for constructing confidence intervals about counterfactual means.

- Example: Y is binary so counterfactual mean is between 0 and 1. We would like our confidence interval to respect this.

We can build an interval on the logit scale and back-transform,

$$\text{expit}\left[\log\left\{\frac{\psi_n(1)}{1 - \psi_n(1)}\right\} \pm z_{1-\alpha/2}\sigma_n^{\text{logit}}\right].$$

```
logitMean <- list(f = function(eff){ qlogis(eff) },  
                 f_inv = function(eff){ plogis(eff) },  
                 h = function(est){ est[1] },  
                 fh_grad = function(est){ c(1/(est[1] - est[1]^2), 0) })  
ci(fit1, contrast = logitMean)
```

```
## $drtmle  
##           est    cil ciu  
## user contrast 0.433 0.368 0.5
```

Basic calls to `drtmle`

Hypothesis tests can be implemented in much the same way using the `wald_test` method.

- `null` specifies what value to test against.

Here, we perform two two-sided hypothesis tests:

```
wald_test(fit1, null = c(0.5, 0.6))
```

```
## $drtmle
##           zstat  pval
## H0: E[Y(0)]=0.5 -1.980 0.048
## H0: E[Y(1)]=0.6  1.162 0.245
```

Basic calls to drtmle

As with `ci`, `wald_test` allows for testing linear combinations of counterfactual means via the `contrast` option.

- We can use this to test the null hypothesis that $ATE = 0$.
- We can test that the ATE equals a particular value via `null` option.

```
# test ATE = 0
wald_test(fit1, contrast = c(1, -1))
```

```
## $drtmle
##              zstat  pval
## H0:E[Y(0)]-E[Y(1)]=0 -3.18 0.001
```

```
# test ATE = 0.1
wald_test(fit1, contrast = c(1, -1), null = 0.1)
```

```
## $drtmle
##              zstat  pval
## H0:E[Y(0)]-E[Y(1)]=0.1 -4.441    0
```

Basic calls to drtmle

`wald_test` also allows testing of arbitrary smooth contrasts of counterfactual means.

We can generally test the null hypothesis that $f(h(\psi_0))$ equals f_0 (the function f applied to the value passed to `null`) using the Wald statistic

$$Z_n := \frac{f(h(\psi_n)) - f_0}{\{\nabla f(h(\psi_n))^T \Sigma_n \nabla f(h(\psi_n))\}^{1/2}} .$$

We can use the `riskRatio` contrast defined previously to test $H_0 : E[Y(0)]/E[Y(1)] = 1$.

```
wald_test(fit1, contrast = riskRatio, null = 1)
```

```
## $drtmle
##               zstat pval
## H0: user contrast = 1 -3.541    0
```

Note on common warning messages

You may see the following warning messages:

```
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
```

- Indicates that some logistic regression has lead to predicted values of 0 or 1.
- May indicate over-fitting of a regression, but generally for our purposes safe to ignore.

```
## Warning in method$computeCoef(Z = Z, Y = Y, libraryNames  
## = libraryNames, : SL.glm_All are duplicates of previous  
## learners. Removing from super learner.
```

- Indicates that two regressions had an identical fit in SuperLearner.
- E.g., SL.step chooses the intercept-only model, which is the same as SL.mean.
- The duplicated algorithm is dropped from the super learner.

Note on the `SL_Qr` and `SL_gr`

Additional residual regressions are needed for doubly-robust inference.

- How these extra regressions are fit is specified via `SL_Qr` and `SL_gr`.

It is difficult to know a-priori how these regressions should be fit. Thus, we recommend being as flexible as possible in their estimation.

- A sensible “default” library might include `SL.glm`, `SL.gam`, `SL.mean`, `SL.earth`, and `SL.npreg`.
- `SL.npreg` is kernel regression with cross-validated bandwidth selection (discussed in Chapter 3).

Note on custom regression estimators

It is not necessary to use super learning to estimate regressions in `drtmle`.

- `glm_` options may be used to fit these regressions (see documentation).

For example, say that to estimate the outcome regression, we would like to use the screening wrapper that we wrote in Lab 1 and

- screen for variables that change the coefficient of A more than 10%;
- fit logistic regression with the resulting variables.

Users can write their own wrapper function that is passed to `SL_` option.

- Format of function identical to what is used by `SuperLearner` (see Lab 1).

Note on custom regression estimators

Here is the same screening function from Lab 1.

```
screen_confounders <- function(Y, X, family, trt_name = "A",
                               change = 0.1, ...){
  # fit treatment only model & get coefficient for treatment
  fit_init <- glm(as.formula(paste0("Y ~ ", trt_name)),
                 data = X, family = family)
  trt_coef <- fit_init$coef[2]
  # identify which column of X is the trt variable
  trt_col <- which(colnames(X) == trt_name)
  # set all variables except trt to not be included initially
  include <- rep(FALSE, ncol(X)); include[trt_col] <- TRUE
  # loop over variables in X other than trt
  for(j in seq_len(ncol(X))[-trt_col]){
    # find variable name
    var_name <- colnames(X)[j]
    # fit trt + variable model, get trt coefficient
    fit <- glm(as.formula(paste0("Y ~ ", trt_name, "+", var_name)),
              data = X, family = family)
    new_trt_coef <- fit$coef[2]
    # check if changed more than specified amount
    include[j] <- abs((new_trt_coef - trt_coef)/trt_coef) > change
  }
  return(include)
}
```

Note on custom regression estimators

Now we write a SuperLearner-style wrapper that does what we want.

```
SL.screened_regression <- function(Y, X, newX, family, ...){  
  # screen columns of X using screen_confounders  
  include_cols <- screen_confounders(Y = Y, X = X, family = family)  
  # fit main terms glm with only those columns  
  fitted_glm <- glm(Y ~ ., data = X[, include_cols], family = family)  
  # get predictions  
  pred <- predict(fitted_glm, newdata = newX[, include_cols],  
                  type = "response")  
  # format output  
  out <- list(fit = list(fitted_model = fitted_glm,  
                        include_cols = include_cols),  
             pred = pred)  
  # assign class  
  class(out$fit) <- "SL.screened_regression"  
  return(out)  
}
```

Note on custom regression estimators

We also need to define a predict method.

```
predict.SL.screened_regression <- function(object, newdata, ...){  
  pred <- predict(object$fitted_model,  
                  newdata = newdata[, object$include_cols],  
                  type = "response")  
  return(pred)  
}
```

Note on custom regression estimators

We can now use `drtmle` with a main-terms logistic regression for the propensity and our custom outcome regression wrapper.

```
set.seed(123)
fit2 <- drtmle(W = W, A = A, Y = Y, a_0 = c(0,1), family = gaussian(),
  # specify main terms logistic regression via glm_g
  glm_g = "W1 + W2",
  # specify custom outcome regression via SL_Qs
  SL_Q = "SL.screened_regression",
  # the residual regression stay the same
  SL_gr = c("SL.earth", "SL.glm", "SL.mean"),
  SL_Qr = c("SL.earth", "SL.glm", "SL.mean"),
  stratify = FALSE)

fit2

## $est
##
## 0 0.4309165
## 1 0.6927319
##
## $cov
##           0           1
## 0 0.0011221610 0.0001007745
## 1 0.0001007745 0.0053828751
```

Example write-up of TMLE analysis

Methods

We estimated the average counterfactual outcome if patients received treatment versus if patients received control using super learning and targeted minimum loss-based estimation with robust inference (Benkeser et al. 2017). This procedure requires regression of the outcome on treatment and confounders and of the treatment on confounders. The set of putative confounders included in these regressions included [...]. For the outcome regressions, we estimated the linear combination of candidate regression estimators that minimizes ten-fold cross-validated mean squared-error. We included three candidate regression estimators in the super learner: polynomial multivariate regression splines, main terms logistic regression, and intercept-only regression. The same set of candidate estimators was used for estimating the probability of treatment. However, in this case we estimated the logistic-linear combination of regression estimators that minimizes ten-fold cross-validated negative log-likelihood loss. To produce robust inference, this procedure additionally requires residual smoothing, which was also achieved via super learning (details in Appendix A). We tested the null hypothesis that the average outcomes were the same under treatment versus control using a two-sided, level 0.05 Wald test with robust influence function-based standard errors estimates (Benkeser et al. 2017). Analyses were performed using the SuperLearner and drtmle R packages (Polley et al, 2018; Lendle et al 2017).

Example write-up of TMLE analysis

Results

The super learners for the outcome regression gave weight to several algorithms with the majority of the weight placed on polynomial multivariate adaptive regression splines, while for the treatment probability the main-terms logistic regression received the most weight the most weight (Table 1, Appendix A).

The estimated average counterfactual outcome if patients received treatment at all three time points was ... (95% CI: ..., ...). The estimated average counterfactual outcome if patients received control at all three time points was ... (95% CI: ..., ...). Our test of the null hypothesis of equality of these quantities rejected the null hypothesis (p -value = ...).